

```

    }
  }
  object IO {
    def empty: IO = new IO { def run = () }
  }

```

The only thing we can perhaps say about `IO` as it stands right now is that it forms a *Monoid* (`empty` is the identity, and `++` is the associative operation). So if we have, for example, a `List[IO]`, we can reduce that to a single `IO`, and the associativity of `++` means that we can do this either by folding left or folding right. On its own, this isn't very interesting. All it seems to have given us is the ability to delay when a side effect actually happens.

Now we'll let you in on a secret: you, as the programmer, get to invent whatever API you wish to represent your computations, including those that interact with the universe external to your program. This process of crafting pleasing, useful, and composable descriptions of what you want your programs to do is at its core *language design*. You're crafting a little language, and an associated *interpreter*, that will allow you to express various programs. If you don't like something about this language you've created, change it! You should approach this like any other design task.

13.2.1 Handling input effects

As you've seen before, sometimes when building up a little language you'll encounter a program that it can't express. So far our `IO` type can represent only "output" effects. There's no way to express `IO` computations that must, at various points, wait for input from some external source. Suppose we wanted to write a program that prompts the user for a temperature in degrees Fahrenheit, and then converts this value to Celsius and echoes it to the user. A typical imperative program might look something like this.¹

Listing 13.2 Imperative program that converts Fahrenheit to Celsius

```

def fahrenheitToCelsius(f: Double): Double =
  (f - 32) * 5.0/9.0

def converter: Unit = {
  println("Enter a temperature in degrees Fahrenheit: ")
  val d = readLine.toDouble
  println(fahrenheitToCelsius(d))
}

```

Unfortunately, we run into problems if we want to make `converter` into a pure function that returns an `IO`:

```

def fahrenheitToCelsius(f: Double): Double =
  (f - 32) * 5.0/9.0

def converter: IO = {
  val prompt: IO = PrintLine(

```

¹ We're not doing any sort of error handling here. This is just meant to be an illustrative example.

```

    "Enter a temperature in degrees Fahrenheit: ")
  // now what ???
}

```

In Scala, `readLine` is a `def` with the side effect of capturing a line of input from the console. It returns a `String`. We could wrap a call to `readLine` in `IO`, but we have nowhere to put the result! We don't yet have a way of representing this sort of effect. The problem is that our current `IO` type can't express computations that *yield a value* of some meaningful type—our interpreter of `IO` just produces `Unit` as its output. Should we give up on our `IO` type and resort to using side effects? Of course not! We extend our `IO` type to allow *input*, by adding a type parameter:

```

sealed trait IO[A] { self =>
  def run: A
  def map[B](f: A => B): IO[B] =
    new IO[B] { def run = f(self.run) }
  def flatMap[B](f: A => IO[B]): IO[B] =
    new IO[B] { def run = f(self.run).run }
}

```

An `IO` computation can now return a meaningful value. Note that we've added `map` and `flatMap` functions so `IO` can be used in for-comprehensions. And `IO` now forms a `Monad`:

```

object IO extends Monad[IO] {
  def unit[A](a: => A): IO[A] = new IO[A] { def run = a }
  def flatMap[A,B](fa: IO[A])(f: A => IO[B]) = fa flatMap f
  def apply[A](a: => A): IO[A] = unit(a)
}

```

This method lets us use the function application syntax to construct `IO` blocks, as in `IO { ... }`.

We can now write our converter example:

```

def ReadLine: IO[String] = IO { readLine }
def PrintLine(msg: String): IO[Unit] = IO { println(msg) }

def converter: IO[Unit] = for {
  _ <- PrintLine("Enter a temperature in degrees Fahrenheit: ")
  d <- ReadLine.map(_.toDouble)
  _ <- PrintLine(fahrenheitToCelsius(d).toString)
} yield ()

```

Our converter definition no longer has side effects—it's a referentially transparent description of a computation with effects, and `converter.run` is the interpreter that will actually execute those effects. And because `IO` forms a `Monad`, we can use all the monadic combinators we wrote previously. Here are some other example usages of `IO`:

- `val echo = ReadLine.flatMap(PrintLine)`—An `IO[Unit]` that reads a line from the console and echoes it back
- `val readInt = ReadLine.map(_.toInt)`—An `IO[Int]` that parses an `Int` by reading a line from the console

- `val readInts = readInt ** readInt—An IO[(Int,Int)]` that parses an `(Int,Int)` by reading two lines from the console²
- `replicateM(10)(ReadLine)—An IO[List[String]]` that will read 10 lines from the console and return the list of results³

Let's do a larger example—an interactive program that prompts the user for input in a loop and then computes the factorial of the input. Here's an example run:

```
The Amazing Factorial REPL, v2.0
q - quit
<number> - compute the factorial of the given number
<anything else> - crash spectacularly
3
factorial: 6
7
factorial: 5040
q
```

The code for this is shown in listing 13.3. It uses a few Monad functions we haven't seen yet: `when`, `foreachM`, and `sequence_`, discussed in the sidebar. For the full listing, see the associated chapter code. The details of this code aren't too important; the point here is just to demonstrate how we can embed an imperative programming language into the purely functional subset of Scala. All the usual imperative programming tools are here—we can write loops, perform I/O, and so on.

Listing 13.3 An imperative program with a `doWhile` loop

```
def factorial(n: Int): IO[Int] = for {
  acc <- ref(1)
  _ <- foreachM (1 to n toStream) (i => acc.modify(_ * i).skip)
  result <- acc.get
} yield result

val factorialREPL: IO[Unit] = sequence_(
  IO { println(helpstring) },
  doWhile { IO { readLine } } { line =>
    when (line != "q") { for {
      n <- factorial(line.toInt)
      _ <- IO { println("factorial: " + n) }
    } yield () }
  })
```

Allocation of a mutable reference.

Imperative factorial using a mutable IO reference.

Modify reference in a loop.

Dereference to obtain the value inside a reference.

See sidebar for definition of `doWhile`.

² Recall that `a ** b` is the same as `map2(a, b) ((_, _))`; it combines two effects into a pair of their results.

³ Recall that `replicateM(3)(fa)` is the same as `sequence(List(fa, fa, fa))`.